

Testing — pytest, Test Strategy & Examples

Framework: pytest + pytest-django **Coverage target:** 75%+ backend code coverage

Table of Contents

1. [Test Setup](#)
 2. [Test Structure](#)
 3. [Model Tests](#)
 4. [API View Tests](#)
 5. [Service Layer Tests](#)
 6. [WebSocket Consumer Tests](#)
 7. [Celery Task Tests](#)
 8. [Test Factories](#)
 9. [Running Tests](#)
-

1. Test Setup

```
# pytest.ini
[pytest]
DJANGO_SETTINGS_MODULE = config.settings.development
python_files = tests/test_*.py
python_classes = Test*
python_functions = test_*
addopts =
    --reuse-db
    --strict-markers
    -v
    --tb=short
markers =
    slow: marks tests as slow (deselect with -m "not slow")
    integration: marks integration tests
    websocket: marks WebSocket tests

# conftest.py (project root)
```

```
import pytest
from django.contrib.gis.geos import Point
from rest_framework.test import APIClient
```

```
@pytest.fixture
def api_client():
    return APIClient()
```

```
@pytest.fixture
def user(db):
    from apps.users.models import UserProfile
    return UserProfile.objects.create_user(
        phone_number='+919876543210',
        full_name='Test User',
        role='USER',
    )
```

```
@pytest.fixture
def guard_user(db):
    from apps.users.models import UserProfile
    from apps.guards.models import GuardProfile
    user = UserProfile.objects.create_user(
        phone_number='+919876543211',
        full_name='Test Guard',
        role='GUARD',
    )
    guard = GuardProfile.objects.create(
        user=user,
        guard_type='UNARMED',
        verification_status='ACTIVE',
        is_online=True,
        current_location=Point(77.5946, 12.9716, srid=4326),
        average_rating=4.5,
    )
    return user
```

```
@pytest.fixture
def auth_client(api_client, user):
    """API client authenticated as a regular user."""
    from rest_framework_simplejwt.tokens import RefreshToken
    refresh = RefreshToken.for_user(user)
    api_client.credentials(HTTP_AUTHORIZATION=f'Bearer {str(refresh.access_token)}')
    api_client.user = user
    return api_client
```

```
@pytest.fixture
def guard_auth_client(api_client, guard_user):
```

```

"""API client authenticated as a guard."""
from rest_framework_simplejwt.tokens import RefreshToken
refresh = RefreshToken.for_user(guard_user)
api_client.credentials(HTTP_AUTHORIZATION=f'Bearer
    {str(refresh.access_token)}')
api_client.user = guard_user
return api_client

@pytest.fixture
def admin_client(api_client, db):
    """API client authenticated as admin."""
    from apps.users.models import UserProfile
    admin = UserProfile.objects.create_superuser(
        phone_number='+919876543299',
        password='adminpass',
    )
    from rest_framework_simplejwt.tokens import RefreshToken
    refresh = RefreshToken.for_user(admin)
    api_client.credentials(HTTP_AUTHORIZATION=f'Bearer
        {str(refresh.access_token)}')
    return api_client

```

2. Test Structure

```

apps/
├── bookings/
│   └── tests/
│       ├── __init__.py
│       ├── conftest.py           # App-specific fixtures
│       └── test_models.py        # Model field validation, FSM
transitions
├── test_views.py                 # API endpoint tests (request/
response)
├── test_services.py              # Business logic tests
└── test_tasks.py                 # Celery task tests

```

3. Model Tests

```

# apps/bookings/tests/test_models.py

import pytest
from django.utils import timezone
from datetime import timedelta
from django_fsm import TransitionNotAllowed

@pytest.mark.django_db

```

```

class TestBookingFSM:

    def setup_method(self):
        from tests.factories import BookingFactory
        self.booking = BookingFactory(status='REQUESTED')

    def test_requested_to_broadcast(self):
        self.booking.start_broadcast()
        self.booking.save()
        assert self.booking.status == 'BROADCAST'

    def test_broadcast_to_accepted(self, guard_user):
        self.booking.start_broadcast()
        self.booking.guard_accept(guard_user.guard_profile)
        assert self.booking.status == 'ACCEPTED'
        assert self.booking.guard == guard_user.guard_profile
        assert self.booking.guard_accepted_at is not None

    def test_invalid_transition_raises_error(self):
        """Cannot go from REQUESTED directly to ACTIVE."""
        with pytest.raises(TransitionNotAllowed):
            self.booking.start_session()

    def test_cancel_from_accepted(self, user):
        self.booking.start_broadcast()
        self.booking.cancel(cancelled_by=user, reason='Changed my mind')
        assert self.booking.status == 'CANCELLED'
        assert self.booking.cancellation_reason == 'Changed my mind'

    def test_full_happy_path(self, guard_user, user):
        """Test complete booking lifecycle from request to completion."""
        booking = self.booking

        booking.start_broadcast()
        booking.guard_accept(guard_user.guard_profile)
        booking.guard_start_travel()
        booking.guard_arrive()
        booking.start_session()

        assert booking.session_started_at is not None

        booking.complete_session()

        assert booking.status == 'COMPLETED'
        assert booking.session_ended_at is not None

```

@pytest.mark.django_db

```

class TestBookingOTP:

```

```

def test_generate_and_verify_start_otp(self):
    from tests.factories import BookingFactory
    booking = BookingFactory(status='ARRIVED')

    otp = booking.generate_start_otp()

    assert len(otp) == 4
    assert otp.isdigit()
    assert booking.verify_start_otp(otp) is True
    assert booking.verify_start_otp('0000') is False

def test_otp_hash_stored_not_plaintext(self):
    from tests.factories import BookingFactory
    booking = BookingFactory()
    otp = booking.generate_start_otp()
    assert booking.start_otp_hash != otp
    assert len(booking.start_otp_hash) == 64 # SHA-256 hex
    digest

```

4. API View Tests

apps/authentication/tests/test_views.py

```

import pytest
from unittest.mock import patch, MagicMock

```

```

@pytest.mark.django_db
class TestSendOTPView:

```

```

def test_send_otp_success(self, api_client):
    with
    patch('apps.notifications.tasks.send_otp_sms.apply_async')
    as mock_task:
        response = api_client.post('/api/auth/send-otp/', {
            'phone_number': '+919876543210',
            'role': 'USER',
        })

    assert response.status_code == 200
    assert response.data['data']['message'] == 'OTP sent
    successfully'
    assert response.data['data']['expires_in'] == 300
    mock_task.assert_called_once()

def test_send_otp_invalid_phone(self, api_client):
    response = api_client.post('/api/auth/send-otp/', {
        'phone_number': '1234', # Invalid format
        'role': 'USER',
    })
    assert response.status_code == 400

```

```

    assert response.data['error']['code'] ==
        'VALIDATION_ERROR'

def test_send_otp_rate_limited(self, api_client):
    """After 5 OTPs in 10 min, should get 429."""
    with
        patch('apps.notifications.tasks.send_otp_sms.apply_async'):
            for _ in range(5):
                api_client.post('/api/auth/send-otp/',
                                {'phone_number': '+919876543210', 'role': 'USER'})

                response = api_client.post('/api/auth/send-otp/',
                                {'phone_number': '+919876543210', 'role': 'USER'})

    assert response.status_code == 429
    assert response.data['error']['code'] ==
        'RATE_LIMIT_EXCEEDED'

@pytest.mark.django_db
class TestVerifyOTPView:

    def test_verify_otp_creates_new_user(self, api_client):
        from apps.authentication.models import OTPToken
        from django.utils import timezone
        from datetime import timedelta
        import hashlib

        otp_code = '123456'
        OTPToken.objects.create(
            phone_number='+919876543210',

            otp_hash=hashlib.sha256(otp_code.encode()).hexdigest(),
            expires_at=timezone.now() + timedelta(minutes=5),
        )

        response = api_client.post('/api/auth/verify-otp/', {
            'phone_number': '+919876543210',
            'otp_code': otp_code,
            'role': 'USER',
        })

        assert response.status_code == 200
        data = response.data['data']
        assert 'access' in data
        assert 'refresh' in data
        assert data['is_new_user'] is True

    def test_verify_otp_wrong_code(self, api_client):
        from apps.authentication.models import OTPToken
        from django.utils import timezone
        from datetime import timedelta
        import hashlib

```

```

OTPToken.objects.create(
    phone_number='+919876543210',

    otp_hash=hashlib.sha256('999999'.encode()).hexdigest(),
    expires_at=timezone.now() + timedelta(minutes=5),
)

response = api_client.post('/api/auth/verify-otp/', {
    'phone_number': '+919876543210',
    'otp_code': '000000',
})

assert response.status_code == 400
assert 'INVALID_OTP' in response.data['error']['code']

```

@pytest.mark.django_db

class TestBookingAPI:

```

def test_create_booking_success(self, auth_client,
    guard_user):
    from django.utils import timezone
    from datetime import timedelta
    from unittest.mock import patch

    # Create wallet with sufficient balance
    from apps.payments.models import Wallet
    Wallet.objects.create(user=auth_client.user, balance=1000)

    start = timezone.now() + timedelta(hours=1)
    end = start + timedelta(hours=3)

    with
    patch('apps.bookings.tasks.broadcast_booking_request.apply_async'):
        response = auth_client.post('/api/bookings/', {
            'service_type': 'HOURLY',
            'guard_type_requested': 'UNARMED',
            'scheduled_start': start.isoformat(),
            'scheduled_end': end.isoformat(),
            'is_immediate': False,
            'service_latitude': 12.9716,
            'service_longitude': 77.5946,
            'service_address': 'Test Address, Bengaluru',
        })

    assert response.status_code == 201
    assert response.data['data']['status'] == 'REQUESTED'

def test_create_booking_insufficient_balance(self,
    auth_client):
    from apps.payments.models import Wallet
    Wallet.objects.create(user=auth_client.user, balance=10)
    # Too low

```

```

from django.utils import timezone
from datetime import timedelta
start = timezone.now() + timedelta(hours=1)
end = start + timedelta(hours=3)

response = auth_client.post('/api/bookings/', {
    'service_type': 'HOURLY',
    'guard_type_requested': 'UNARMED',
    'scheduled_start': start.isoformat(),
    'scheduled_end': end.isoformat(),
    'is_immediate': False,
    'service_latitude': 12.9716,
    'service_longitude': 77.5946,
    'service_address': 'Test Address',
})

assert response.status_code == 400
assert response.data['error']['code'] ==
    'INSUFFICIENT_BALANCE'

def test_unauthenticated_booking_rejected(self, api_client):
    response = api_client.post('/api/bookings/', {})
    assert response.status_code == 401

```

5. Service Layer Tests

apps/payments/tests/test_services.py

```

import pytest
from decimal import Decimal

@pytest.mark.django_db
class TestWalletService:

    def test_credit_increases_balance(self, user):
        from apps.payments.models import Wallet
        from apps.payments.services import WalletService

        wallet = Wallet.objects.create(user=user,
            balance=Decimal('100.00'))

        txn = WalletService.credit(
            user=user,
            amount=Decimal('250.00'),
            transaction_type='TOPUP',
        )

        wallet.refresh_from_db()
        assert wallet.balance == Decimal('350.00')

```



```

    assert txn.balance_after == Decimal('350.00')
    assert txn.transaction_type == 'TOPUP'
    assert txn.status == 'SUCCESS'

def test_debit_decreases_balance(self, user):
    from apps.payments.models import Wallet
    from apps.payments.services import WalletService

    Wallet.objects.create(user=user,
        balance=Decimal('500.00'))

    WalletService.debit(user, Decimal('200.00'),
        'BOOKING_DEBIT')

    wallet = user.wallet
    wallet.refresh_from_db()
    assert wallet.balance == Decimal('300.00')

def test_debit_raises_on_insufficient_balance(self, user):
    from apps.payments.models import Wallet
    from apps.payments.services import WalletService,
        InsufficientBalanceError

    Wallet.objects.create(user=user, balance=Decimal('50.00'))

    with pytest.raises(InsufficientBalanceError):
        WalletService.debit(user, Decimal('200.00'),
            'BOOKING_DEBIT')

def test_concurrent_debits_are_safe(self, user):
    """Verify SELECT FOR UPDATE prevents race conditions."""
    import threading
    from apps.payments.models import Wallet
    from apps.payments.services import WalletService

    Wallet.objects.create(user=user,
        balance=Decimal('300.00'))
    errors = []

    def debit_200():
        try:
            WalletService.debit(user, Decimal('200.00'),
                'BOOKING_DEBIT')
        except Exception as e:
            errors.append(e)

    threads = [threading.Thread(target=debit_200) for _ in
        range(2)]
    for t in threads:
        t.start()
    for t in threads:
        t.join()

```

```

# Exactly one should succeed, one should raise
InsufficientBalanceError
user.wallet.refresh_from_db()
assert user.wallet.balance == Decimal('100.00') # Only
one debit went through
assert len(errors) == 1

@pytest.mark.django_db
class TestPricingEngine:

    def test_hourly_pricing(self):
        from apps.payments.pricing import PricingEngine
        from django.utils import timezone
        from datetime import timedelta

        start = timezone.now()
        end = start + timedelta(hours=3)

        result = PricingEngine.estimate_price('UNARMED', start,
        end)

        assert result['duration_hours'] == 3.0
        assert result['base_rate_per_hour'] == 150.0
        assert result['total_amount'] == pytest.approx(531.0,
        rel=0.01)

    def test_minimum_2_hours_enforced(self):
        from apps.payments.pricing import PricingEngine
        from django.utils import timezone
        from datetime import timedelta

        start = timezone.now()
        end = start + timedelta(hours=1) # Only 1 hour

        result = PricingEngine.estimate_price('UNARMED', start,
        end)

        # Should be billed for minimum 2 hours
        assert result['duration_hours'] == 2.0

```

6. WebSocket Consumer Tests

```

# apps/tracking/tests/test_consumers.py

import pytest
import json
from channels.testing import WebsocketCommunicator
from channels.layers import get_channel_layer
from asgiref.sync import sync_to_async

```

```

@pytest.mark.asyncio
@pytest.mark.django_db(transaction=True)
class TestTrackingConsumer:

    async def test_unauthenticated_connection_rejected(self):
        from config.asgi import application

        communicator = WebsocketCommunicator(
            application,
            '/ws/tracking/00000000-0000-0000-0000-000000000001/'
            # No ?token= param
        )
        connected, code = await communicator.connect()
        assert not connected
        assert code == 4001

    async def test_guard_can_send_location(self, booking,
        guard_token):
        from config.asgi import application

        booking_id = str(booking.id)
        url = f'/ws/tracking/{booking_id}/?token={guard_token}'

        communicator = WebsocketCommunicator(application, url)
        connected, _ = await communicator.connect()
        assert connected

        # Send location update
        await communicator.send_json_to({
            'type': 'location_update',
            'lat': 12.9716,
            'lng': 77.5946,
            'accuracy': 5.0,
        })

        # No error should be received
        response = await communicator.receive_json_from()
        # Should get the broadcast back
        assert response['type'] == 'guard_location'
        assert response['lat'] == 12.9716

        await communicator.disconnect()

    async def test_user_receives_guard_location(self, booking,
        guard_token, user_token):
        """Both user and guard join the session group; user
        receives guard's location."""
        from config.asgi import application

        booking_id = str(booking.id)

        guard_comm = WebsocketCommunicator(

```

```

        application,
        f'/ws/tracking/{booking_id}?token={guard_token}'
    )
    user_comm = WebsocketCommunicator(
        application,
        f'/ws/tracking/{booking_id}?token={user_token}'
    )

    await guard_comm.connect()
    await user_comm.connect()

    # Consume the initial session_state messages
    await guard_comm.receive_json_from()
    await user_comm.receive_json_from()

    # Guard sends location
    await guard_comm.send_json_to({
        'type': 'location_update',
        'lat': 12.9716,
        'lng': 77.5946,
    })

    # User should receive it
    message = await user_comm.receive_json_from()
    assert message['type'] == 'guard_location'
    assert message['lat'] == 12.9716

    await guard_comm.disconnect()
    await user_comm.disconnect()

```

7. Celery Task Tests

apps/bookings/tests/test_tasks.py

```

import pytest
from unittest.mock import patch, MagicMock

@pytest.mark.django_db
class TestBroadcastBookingRequest:

    def test_sends_push_to_nearby_guards(self):
        from apps.bookings.tasks import broadcast_booking_request
        from tests.factories import BookingFactory,
        GuardProfileFactory
        from django.contrib.gis.geos import Point

        booking = BookingFactory(
            status='REQUESTED',
            service_latitude=12.9716,

```

```

        service_longitude=77.5946,
        guard_type_requested='UNARMED',
    )

    # Create a guard 2km away
    guard = GuardProfileFactory(
        verification_status='ACTIVE',
        is_online=True,
        guard_type='UNARMED',
        current_location=Point(77.6100, 12.9800, srid=4326),
    )
    guard.user.fcm_token = 'test_fcm_token_123'
    guard.user.save()

    with
    patch('apps.notifications.services.push.FCMService.send_to_device')
    as mock_fcm:
        broadcast_booking_request(str(booking.id),
            radius_km=5)

    mock_fcm.assert_called_once()
    call_args = mock_fcm.call_args
    assert call_args[0][0] == 'test_fcm_token_123'

def test_expands_radius_if_no_guards(self):
    from apps.bookings.tasks import broadcast_booking_request
    from tests.factories import BookingFactory

    booking = BookingFactory(
        status='REQUESTED',
        service_latitude=12.9716,
        service_longitude=77.5946,
    )

    with
    patch('apps.bookings.tasks.broadcast_booking_request.apply_async')
    as mock_retry:
        broadcast_booking_request(str(booking.id),
            radius_km=5)

    # Should schedule retry with expanded radius
    mock_retry.assert_called_once()
    args, kwargs = mock_retry.call_args
    assert args[0] == [str(booking.id), 10] # Expanded to
    10km

```

8. Test Factories

tests/factories.py

import factory

```

import factory.fuzzy
from django.contrib.gis.geos import Point
from django.utils import timezone
from datetime import timedelta

class UserProfileFactory(factory.django.DjangoModelFactory):
    class Meta:
        model = 'users.UserProfile'

    phone_number = factory.Sequence(lambda n: f'+9198765{n:05d}')
    full_name = factory.Faker('name', locale='en_IN')
    role = 'USER'
    is_active = True

class GuardProfileFactory(factory.django.DjangoModelFactory):
    class Meta:
        model = 'guards.GuardProfile'

    user = factory.SubFactory(UserProfileFactory, role='GUARD')
    guard_type = 'UNARMED'
    verification_status = 'ACTIVE'
    is_online = True
    current_location = factory.LazyFunction(lambda:
        Point(77.5946, 12.9716, srid=4326))
    average_rating = factory.fuzzy.FuzzyDecimal(3.5, 5.0,
        precision=2)

class BookingFactory(factory.django.DjangoModelFactory):
    class Meta:
        model = 'bookings.Booking'

    user = factory.SubFactory(UserProfileFactory)
    guard = None
    service_type = 'HOURLY'
    guard_type_requested = 'UNARMED'
    status = 'REQUESTED'
    scheduled_start = factory.LazyFunction(lambda: timezone.now()
        + timedelta(hours=1))
    scheduled_end = factory.LazyFunction(lambda: timezone.now() +
        timedelta(hours=4))
    is_immediate = True
    service_address = factory.Faker('address')
    service_latitude = factory.fuzzy.FuzzyDecimal(12.9, 13.1,
        precision=6)
    service_longitude = factory.fuzzy.FuzzyDecimal(77.5, 77.7,
        precision=6)
    base_rate_per_hour = factory.fuzzy.FuzzyDecimal(150, 600,
        precision=2)
    surge_multiplier = 1.00

```

```
class WalletFactory(factory.django.DjangoModelFactory):
    class Meta:
        model = 'payments.Wallet'

    user = factory.SubFactory(UserProfileFactory)
    balance = factory.fuzzy.FuzzyDecimal(0, 5000, precision=2)
```

9. Running Tests

Run all tests

```
pytest
```

Run with coverage report

```
pytest --cov=apps --cov-report=html --cov-report=term-missing
```

Run only fast tests (exclude slow/integration)

```
pytest -m "not slow and not integration"
```

Run specific app tests

```
pytest apps/bookings/
```

Run specific test class

```
pytest apps/bookings/tests/test_services.py::TestWalletService
```

Run specific test

```
pytest apps/bookings/tests/
    test_views.py::TestBookingAPI::test_create_booking_success
```

Run WebSocket tests only

```
pytest -m websocket
```

Run with verbose output (see print statements)

```
pytest -s -v
```

Run and stop on first failure

```
pytest -x
```

Run in parallel (faster for large test suites)

```
pip install pytest-xdist
```

```
pytest -n 4 # 4 parallel workers
```

Generate HTML coverage report

```
pytest --cov=apps --cov-report=html
```

```
open htmlcov/index.html
```

Coverage report example output:

```
----- coverage: 81% -----
apps/authentication/services.py    98%
apps/authentication/views.py      95%
apps/bookings/models.py            89%
```

apps/bookings/services.py	82%	
apps/bookings/views.py	91%	
apps/payments/services.py	86%	
apps/tracking/consumers.py	74%	
apps/sos/services.py	79%	
apps/notifications/tasks.py	68%	← needs more tests